UNIVERSITE SAAD DAHLAB — BLIDA 1
Faculte des Sciences — Departement d'Informatique
Module : NoSQL pour le Big Data | M1 DS&NLP | 2025/2026

RAPPORT TECHNIQUE

Systeme de Suivi IoT Temps Reel
avec Apache Cassandra

Etudiant	Abdelaziz Merzoug
Enseignant	Dr. B. BENCHARIF
Module	NoSQL pour le Big Data
Universite	Universite Saad Dahlab — Blida 1
Filiere	M1 DS&NLP
Annee universitaire	2025/2026

1. Introduction

L'emergence de l'Internet des Objets (IoT) a engendre une explosion sans precedent du volume de donnees generees en temps reel. Dans le secteur de la logistique, chaque vehicule equipe de capteurs GPS, de thermometres et d'accelerometres peut produire plusieurs centaines de points de donnees par heure. A l'echelle d'une flotte industrielle de 10 000 camions actifs en permanence, on atteint rapidement plusieurs milliards d'enregistrements par annee — un volume que les systemes de gestion de bases de donnees relationnelles traditionnels (SGBDR) sont fondamentalement incapables de gerer efficacement.

La problematique est double : premierement, les SGBDR tels que MySQL ou PostgreSQL reposent sur un modele transactionnel ACID et une architecture centralisee qui constitue un goulot d'etranglement des lors que le debit d'ecriture depasse quelques milliers d'operations par seconde. Deuxiemement, meme des solutions NoSQL orientees documents comme MongoDB, bien que plus flexibles, presentent des limitations architecturales critiques pour les charges de travail IoT a haute frequence d'ecriture, notamment en raison de leur architecture Primary-Secondary et de leur moteur de stockage B-Tree. Ces limitations seront analysees en detail dans les sections suivantes.

Le present projet implemente un systeme de suivi logistique IoT full-stack simulant dix camions (TRUCK_001 a TRUCK_010) qui transmettent leurs positions GPS, temperatures de cargaison et statuts operationnels vers une base de donnees Apache Cassandra 4.1. Le systeme a ingere avec succes 10 746 enregistrements au cours de la session de simulation, via un script Python multi-thread utilisant exclusivement des Prepared Statements. Un backend FastAPI expose les donnees via six endpoints REST, et un dashboard Streamlit offre une visualisation en temps reel sur une carte centree sur l'Algerie.

2. Analyse Architecturale de Cassandra

2.1 Positionnement dans le Theoreme CAP

Le theoreme CAP, formule par Eric Brewer en 2000 et demontre par Gilbert et Lynch en 2002, etablit qu'un systeme de donnees distribue ne peut garantir simultanement que deux des trois proprietes suivantes : la Coherence (Consistency), la Disponibilite (Availability) et la Tolerance au Partitionnement reseau (Partition Tolerance). En situation de partitionnement — c'est-a-dire lorsque des noeuds du cluster ne peuvent plus communiquer entre eux — le systeme doit choisir entre privilegier la coherence ou la disponibilite.

Apache Cassandra est un systeme AP : il garantit la Disponibilite et la Tolerance au Partitionnement, au detriment de la Coherence forte. En pratique, cela signifie que Cassandra adopte le modele d'Eventual Consistency (coherence eventuelle) : toutes les repliques d'une donnee convergeront vers le meme etat dans un delai fini, mais une lecture immediatement apres une ecriture peut retourner une valeur legerement depassee si la replication n'est pas encore completee sur tous les noeuds.

L'un des atouts distinctifs de Cassandra est sa Tunable Consistency — la coherence reglable. Le developpeur peut ajuster le niveau de coherence par requete selon les besoins applicatifs :

- CONSISTENCY ONE : une seule replique doit repondre a l'ecriture ou la lecture. Latence minimale, disponibilite maximale. Ideal pour l'IoT ou la vitesse prime sur la coherence absolue.
- CONSISTENCY QUORUM : la majorite des repliques (n/2 + 1) doit repondre. Compromis equilibre entre performance et coherence. Recommande pour les applications critiques.
- CONSISTENCY ALL : toutes les repliques doivent repondre. Coherence forte mais disponibilite reduite — un seul noeud en panne bloque l'operation.

Dans notre projet, nous avons configure CONSISTENCY ONE pour toutes les operations d'ecriture et de lecture. Ce choix est delibere et justifie par la nature IoT de la charge de travail : la priorite absolue est de ne perdre aucune mesure GPS, meme en cas de defaillance partielle du cluster. Si un noeud Cassandra tombe en panne, les neuf autres camions continuent d'envoyer leurs donnees sans aucune interruption — le coordinateur de requete reroutant automatiquement vers les noeuds disponibles.

Tableau comparatif des positionnements CAP :

Systeme	C	A	P	Use case typique
Apache Cassandra	~	✓	✓	IoT, logs, Time-series, suivi GPS
MongoDB	✓	~	✓	Documents, e-commerce, CMS
MySQL / PostgreSQL	✓	✓	~	Transactions bancaires, ERP, OLTP

2.2 Le Moteur LSM-Tree (Log-Structured Merge-Tree)

La performance d'ecriture exceptionnelle de Cassandra repose sur une structure de donnees fondamentale : le Log-Structured Merge-Tree (LSM-Tree), introduit par O'Neil et al. en 1996. Contrairement aux bases de donnees relationnelles qui utilisent des arbres B (B-Tree) pour organiser les donnees sur disque, le LSM-Tree privilege les ecritures sequentielles en memoire et sur disque, eliminant la necessite de localiser et modifier des enregistrements existants in-place.

Pipeline d'ecriture LSM-Tree dans Cassandra :

1. MemTable (RAM) : Chaque ecriture est d'abord inseree dans une structure en memoire vive appelee MemTable. Cette operation est un simple append en O(1) — aucune recherche prealable n'est necessaire. La MemTable est organisee par Partition Key pour permettre des lectures efficaces.

2. CommitLog (disque sequentiel) : Simultanement, l'ecriture est journalisee de facon sequentielle dans le CommitLog sur disque. Ce journal assure la durabilite des donnees en cas de panne avant que la MemTable ne soit videe. L'ecriture sequentielle sur disque est jusqu'a 100 fois plus rapide que l'ecriture aleatoire.

3. MemTable Immutable : Lorsque la MemTable atteint sa limite de capacite (configurable, par defaut 1/4 du heap JVM), elle est declaree immutable (lecture seule) et une nouvelle MemTable vide prend sa place pour les nouvelles ecritures. La MemTable immutable est mise en file d'attente pour etre ecrite sur disque.

4. SSTable (Sorted String Table) : La MemTable immutable est serialisee sur disque sous forme de SSTable — un fichier immuable contenant les donnees triees par cle de clustering. Une fois ecrite, une SSTable ne peut etre modifiee ; les mises a jour et suppressions sont representees par de nouveaux enregistrements (tombstones pour les suppressions).

5. Compaction : Cassandra execute periodiquement un processus de compaction qui fusionne plusieurs SSTables en une seule, eliminant les valeurs obsoletes et les tombstones. Trois strategies de compaction existent : Size-Tiered (default, optimal pour les ecritures massives), Leveled (optimal pour les lectures frequentes), et Time-Window (optimal pour les series temporelles a TTL).

Comparaison LSM-Tree vs B-Tree :

Les bases de donnees relationnelles (MySQL, PostgreSQL) et MongoDB utilisent le moteur WiredTiger base sur un B-Tree. Ce choix optimise les lectures point-to-point (O(log n)) mais penalise les ecritures : chaque INSERT ou UPDATE necessite de localiser la position exacte dans l'arbre, de charger la page correspondante depuis le disque en memoire, de la modifier, puis de la reecrire — une operation de lecture-modification-ecriture (read-modify-write) inheremment couteuse.

Dans notre cas concret, le script generator.py insere 10 enregistrements par seconde (un par camion toutes les 0,5 a 1,5 secondes). Avec un moteur B-Tree, chaque insertion impliquerait une recherche dans l'arbre pour positionner le nouveau noeud, potentiellement un reequilibrage de l'arbre, et une ecriture aleatoire sur disque. Avec le LSM-Tree de Cassandra, chaque insertion est un simple append en memoire — instantane et O(1) — avant d'etre flush sur disque de facon sequentielle lors de la vidange de la MemTable.

---

3. Comparaison NoSQL : Cassandra vs MongoDB

La question du choix technologique entre Apache Cassandra et MongoDB est recurrente dans les projets Big Data. Cette section analyse quatre axes fondamentaux permettant de justifier objectivement le choix de Cassandra pour un use-case de suivi GPS a grande echelle.

Axe 1 — Debit d'ecriture a l'echelle d'un milliard de points GPS

Cassandra adopte une architecture masterless (peer-to-peer, P2P) : tous les noeuds du cluster sont strictement equivalents. Chaque noeud peut accepter des ecritures et des lectures independamment, sans coordination avec un noeud central. Le client (notre driver Python) peut ecrire sur n'importe quel noeud du cluster, qui devient alors le coordinateur et route la requete vers le(s) noeud(s) proprietaire(s) de la partition. Cette architecture permet une scalabilite lineaire : doubler le nombre de noeuds double le debit d'ecriture, de facon quasi-proportionnelle.

MongoDB repose au contraire sur une architecture Primary-Secondary (replica set). Toutes les ecritures doivent transiter par le noeud Primary — les noeuds Secondary ne servent qu'a la replication et aux lectures optionnelles. Ce design introduit un Single Point of Bottleneck : le Primary devient le goulot

d'etranglement a mesure que le debit d'ecriture augmente. A l'echelle d'un milliard de points GPS, cette limitation est redhibitoire. De plus, si le Primary tombe en panne, une election est declenchee parmi les secondaires, ce qui provoque une indisponibilite de 10 a 30 secondes — un intervalle pendant lequel tous les camions perdent leurs donnees.

Conclusion : pour une flotte de 10 000 camions generant chacun 100 points GPS par heure, soit 1 million de points par heure, l'architecture Cassandra est la seule option viable. Le throughput est configurable en ajoutant simplement des noeuds au cluster — sans requalification de l'application.

Axe 2 — Adequation du modele de donnees

MongoDB utilise un modele document (BSON/JSON) offrant une grande flexibilite de schema : chaque document peut avoir une structure differente, des champs optionnels, des tableaux imbriques. Cette flexibilite est precieuse pour des donnees heterogenes comme des profils utilisateurs, des catalogues produits ou des evenements de natures variees.

Cassandra utilise un schema fixe defini en CQL, proche du SQL mais optimise pour la distribution. Ses types de donnees sont stricts et le schema doit etre concu a l'avance. Cela peut sembler contraignant, mais c'est en realite un avantage pour notre cas d'usage : les donnees GPS sont parfaitement homogenes. Chaque enregistrement de tracking_by_truck contient exactement les memes champs (truck_id, event_time, latitude, longitude, temperature, cargo_status, speed_kmh) pour tous les camions. La flexibilite documentaire de MongoDB n'apporte ici aucune valeur ajoutee, et son absence dans Cassandra n'est pas un handicap.

Axe 3 — Performance de lecture par cle

Les deux systemes offrent des lectures O(1) sur leur cle principale : MongoDB sur l'_id ou un index, Cassandra sur la Partition Key. Cette equivalence de performance brute masque cependant une difference architecturale importante : la culture des deux systemes face aux requetes non-indexees.

MongoDB autorise les full-scans via des requetes sans index (find({temperature: {$gt: 35}})), ce qui est pratique en developpement mais catastrophique en production sur des collections de milliards de documents. La dangerosite de cette souplesse reside dans le fait qu'elle ne se manifeste qu'a grande echelle, quand il est souvent trop tard pour refactorer le schema.

Cassandra, au contraire, interdit explicitement l'instruction ALLOW FILTERING dans les applications bien concues. Une requete qui necessite un scan de toutes les partitions est rejetee par defaut, forcant le developpeur a concevoir un schema qui garantit que chaque requete accede directement a sa partition cible. C'est une contrainte saine qui impose de bonnes pratiques architecturales des la phase de modelisation — exactement comme dans notre projet ou chaque table a ete concue pour repondre a une requete specifique sans scan.

Axe 4 — Disponibilite et tolerance aux pannes

Cassandra ne presente aucun Single Point of Failure (SPOF). Si n noeuds tombent simultanement et que le Replication Factor est suffisant, le cluster continue de fonctionner sans interruption. N'importe quel

noeud survivant peut coordonner n'importe quelle requete. Cette propriete est fondamentale pour l'IoT : un systeme de suivi de flotte ne peut se permettre aucune interruption, sous peine de perdre des donnees critiques.

MongoDB en replica set souffre d'un SPOF au niveau du Primary. En cas de perte du Primary, l'election d'un nouveau Primary prend entre 10 et 30 secondes selon la configuration. Pendant cet intervalle, les ecritures sont bloquees. Pour notre flotte de 10 000 camions emettant 1 mesure par seconde, 20 secondes d'indisponibilite representent 200 000 points GPS perdus — une perte inacceptable pour un systeme de tracking logistique.

Pour l'IoT, la disponibilite prime sur la coherence. Cassandra, en tant que systeme AP selon le theoreme CAP, est le choix architecturalement correct et optimal pour ce use-case.

4. Justification du Modele de Donnees CQL

4.1 Definitions fondamentales

La modelisation des donnees dans Cassandra suit le paradigme Query-Driven Modeling : on conçoit les tables en fonction des requetes que l'interface devra executer, et non en fonction de la normalisation relationnelle. Cette inversion de philosophie est fondamentale pour comprendre les choix de schema de ce projet.

Primary Key : La cle primaire dans Cassandra est composee de deux elements distincts : la Partition Key et les Clustering Columns (optionnelles). La syntaxe PRIMARY KEY (partition_key, clustering_col1, clustering_col2) encode ces deux concepts dans un seul constructeur.

Partition Key : Determine sur quel noeud du cluster la donnee sera physiquement stockee. Cassandra applique la fonction de hachage Murmur3 sur la valeur de la Partition Key pour calculer un token, qui positionne la donnee sur le token ring du cluster. Toutes les lignes partageant la meme Partition Key sont co-localisees sur le meme noeud (et ses repliques) — leur lecture ne sollicite donc qu'un seul noeud, en O(1).

Clustering Column : Determine l'ordre physique des lignes a l'interieur d'une partition, directement dans les fichiers SSTable sur disque. Cet ordre est etabli a l'ecriture et maintenu automatiquement — la lecture recupere les donnees deja triees sans aucun cout de tri supplementaire.

4.2 Justification de la table tracking_by_truck

La requete principale de l'application est : recuperer les 50 dernieres positions GPS du camion TRUCK_001. Cette requete se traduit naturellement en :

```
SELECT * FROM tracking_by_truck WHERE truck_id = 'TRUCK_001' LIMIT 50;
```

Le choix de truck_id comme Partition Key garantit que toutes les positions d'un camion sont co-localisees sur le meme noeud. La clause WHERE truck_id = 'TRUCK_001' est un acces direct a une seule partition — O(1) garanti independamment du nombre total d'enregistrements dans la base.

Le choix de event_time avec CLUSTERING ORDER BY (event_time DESC) signifie que les enregistrements les plus recents sont physiquement stockes en premier dans la SSTable. Un LIMIT 50 lit donc exactement les 50 premieres entrees de la partition — les 50 plus recentes — sans aucune operation de tri cote applicatif. Si l'ordre avait ete ASC (par defaut), recuperer les 50 dernieres entrees aurait necessaire de lire toute la partition et de la trier en memoire, ou d'utiliser ALLOW FILTERING, les deux solutions etant inacceptables en production.

4.3 Justification de la table alerts_by_day

La requete cible est : afficher toutes les alertes de temperature pour la journee du 01/04/2026. La Partition Key alert_date (type DATE, sans composante horaire) groupe toutes les alertes d'une journee sur le meme noeud. La Clustering Column event_time DESC assure que les alertes les plus recentes apparaissent en premier dans le resultat — comportement attendu pour un tableau de bord de monitoring.

La deuxieme Clustering Column truck_id ASC sert de critere de departage : si deux alertes surviennent exactement a la meme milliseconde (theoriquement possible avec 10 camions paralleles), l'ordre reste deterministe et stable dans les SSTables, evitant toute ambiguite dans le resultat retourne au client.

4.4 Justification de la table truck_stats

La table truck_stats utilise une Partition Key composite (truck_id, stat_hour). Ce choix repond a un probleme specifique : si seul truck_id avait ete utilise, toutes les statistiques historiques d'un camion — potentiellement des milliers d'entrees sur des mois d'exploitation — auraient ete regroupees dans une seule partition, creant une partition de taille croissante indefiniment (unbounded partition growth) et un hotspot sur le noeud qui la gere.

En combinant truck_id et stat_hour (format 'TRUCK_001_2026040109'), chaque partition ne contient qu'une seule ligne representant l'agregat statistique d'un camion pour une heure precise. La charge en ecriture est distribuee uniformement sur tous les noeuds, et la requete fourni la cle composite complete pour un acces O(1) direct.

4.5 Pourquoi ALLOW FILTERING est strictement interdit

L'instruction ALLOW FILTERING ordonne a Cassandra d'effectuer un scan complet de toutes les partitions de la table sur tous les noeuds du cluster. Pour une table de 10 000 enregistrements en environnement de developpement, la latence reste acceptable (quelques centaines de millisecondes). A l'echelle d'un milliard de points GPS en production, cette operation mobiliserait l'integralite des ressources reseau et CPU du cluster pendant plusieurs minutes, saturant les autres requetes et provoquant des timeouts en cascade.

Le schema de ce projet a ete concu selon le principe Query-Driven Modeling pour que chaque requete de l'interface fournisse la Partition Key complete dans sa clause WHERE. Cassandra peut ainsi router chaque

requete directement vers le noeud proprietaire de la partition cible, sans aucun scan. C'est pour cette raison qu'ALLOW FILTERING n'apparait nulle part dans le code source — ni dans generator.py, ni dans app.py, ni dans schema.cql.

5. Journal d'Ingestion et Resultats

5.1 Deroulement de la simulation

Le script generator.py a ete lance depuis un terminal Windows PowerShell avec Python 3.12. Apres resolution d'une incompatibilite liee a la suppression du module asyncore en Python 3.12, la solution adoptee a ete d'utiliser GeventConnection comme event loop de remplacement via la bibliotheque gevent. Les trois Prepared Statements ont ete declares une seule fois dans le thread principal avant le lancement des dix threads de simulation.

Chaque thread simule un camion sur une route algerienne fixe : quatre camions sur la Route A (Alger-Blida, lat 36.4-36.7, lon 2.8-3.1), trois sur la Route B (Blida-Medea, lat 36.2-36.4, lon 2.8-2.9), et trois sur la Route C (Alger-Boumerdes, lat 36.7-36.8, lon 3.1-3.6). La temperature de chaque camion est simulee par une marche aleatoire (random walk) avec un delta de ±0,5°C par iteration, garantissant un comportement realiste plutot qu'une distribution uniforme.

5.2 Vitesse d'ingestion observee

Avec un intervalle d'insertion aleatoire entre 0,5 et 1,5 secondes par camion (moyenne : 1 seconde), le debit theorique est de 10 insertions par seconde. La simulation a ete arretee proprement via Ctrl+C apres environ 17 minutes, produisant le bilan suivant :

Camion	Enregistrements	Route	% du total
TRUCK_001	1091	A	10.2%
TRUCK_002	1082	A	10.1%
TRUCK_003	1078	A	10.0%
TRUCK_004	1072	A	10.0%
TRUCK_005	1075	B	10.0%
TRUCK_006	1091	B	10.2%
TRUCK_007	1068	B	9.9%
TRUCK_008	1057	C	9.8%
TRUCK_009	1065	C	9.9%
TRUCK_010	1067	C	9.9%
TOTAL	**10 746**	**—**	**100%**

5.3 Preuve de l'ingestion — Commande de verification

La commande suivante a ete executee dans cqlsh pour confirmer le total d'enregistrements :

```
docker exec -it cassandra_iot cqlsh -e "SELECT count(*) FROM
logistics_ks.tracking_by_truck;"
 count -------  10746 (1 rows)
```

Note : le warning 'Aggregation query used without partition key' est attendu et normal pour un COUNT(*) global sans clause WHERE. Il indique que Cassandra a effectue un scan de toutes les partitions pour compter les enregistrements. Cette commande est uniquement utilisee pour la verification — jamais dans le code applicatif ou dans les Prepared Statements de l'API.

---

6. Conclusion

Ce projet a permis de mettre en oeuvre l'integralite de la chaine de valeur d'un systeme IoT moderne : de la simulation de capteurs GPS a la visualisation temps reel sur tableau de bord interactif, en passant par un schema de base de donnees optimise et une API REST robuste. Les objectifs quantitatifs ont ete depasses avec 10 746 enregistrements ingeres (objectif : 10 000 minimum), distribues equitablement entre les dix camions simules.

Sur le plan theorique, ce projet demontre concretement pourquoi Apache Cassandra est le choix architecturalement superieur a MongoDB ou aux SGBDR pour les charges de travail IoT a haute frequence d'ecriture. L'architecture masterless elimine tout Single Point of Failure et de Bottleneck, le moteur LSM-Tree garantit des ecritures O(1) en memoire, et le paradigme Query-Driven Modeling force des patterns d'acces efficaces en eliminant ALLOW FILTERING des la phase de conception.

Sur le plan pratique, la combinaison FastAPI + Cassandra + Streamlit constitue une stack moderne, performante et deployable en production via Docker Compose. Les Prepared Statements reduisent significativement la charge de parsing sur le cluster, et l'architecture decouplee (Streamlit n'accede jamais directement a Cassandra) garantit la maintenabilite et l'evolutivite du systeme.

Plusieurs perspectives d'evolution sont envisageables pour porter ce systeme a l'echelle industrielle : le deploiement d'un cluster Cassandra multi-noeuds avec NetworkTopologyStrategy pour la replication multi-datacenter, l'integration d'Apache Kafka comme broker de messages entre les capteurs IoT et Cassandra pour absorber les pics de charge, et l'adoption de Cassandra Replication Factor 3 avec CONSISTENCY QUORUM pour les donnees critiques necessitant une coherence forte.

References

- Documentation officielle Apache Cassandra 4.1 : https://cassandra.apache.org/doc/latest/
- DataStax Python Driver 3.29 : https://docs.datastax.com/en/developer/python-driver/3.29/
- O'Neil, P. et al. (1996). The Log-Structured Merge-Tree (LSM-Tree). Acta Informatica, 33(4), 351-385.
- Gilbert, S. & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News, 33(2), 51-59.
- FastAPI Documentation : https://fastapi.tiangolo.com
- Streamlit Documentation : https://docs.streamlit.io
- Docker Compose Reference : https://docs.docker.com/compose/
- Gevent Library : https://www.gevent.org/